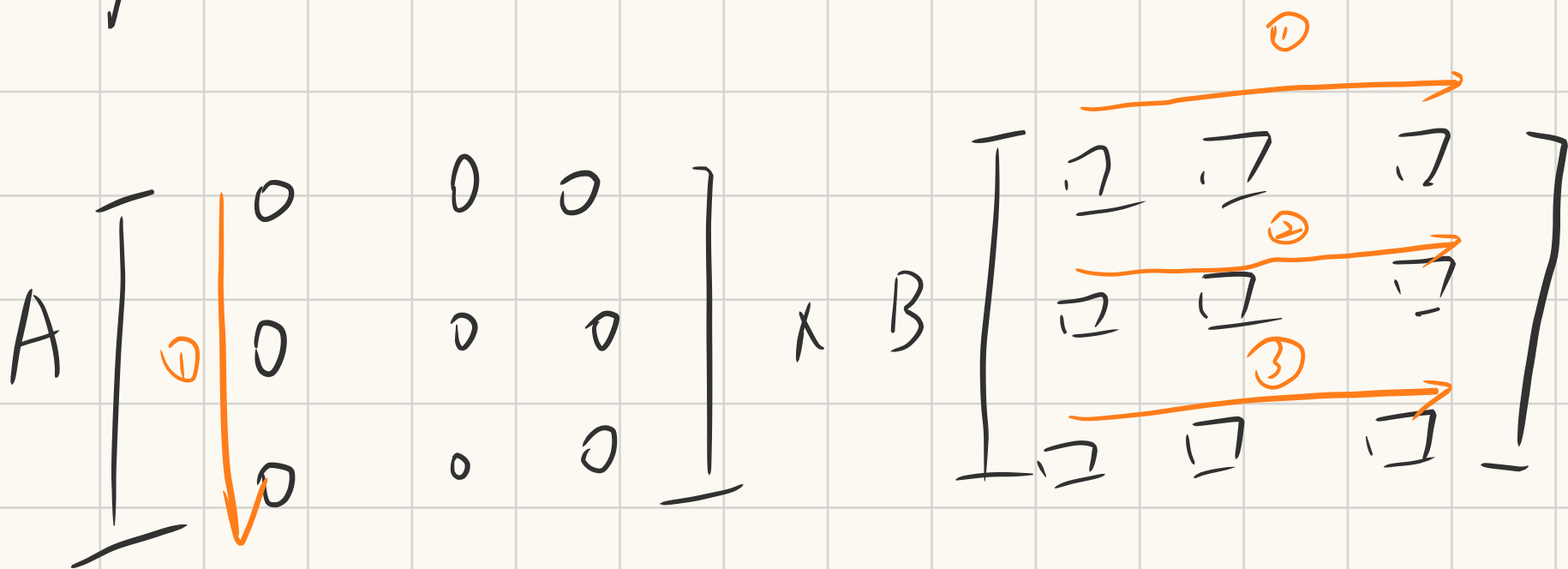


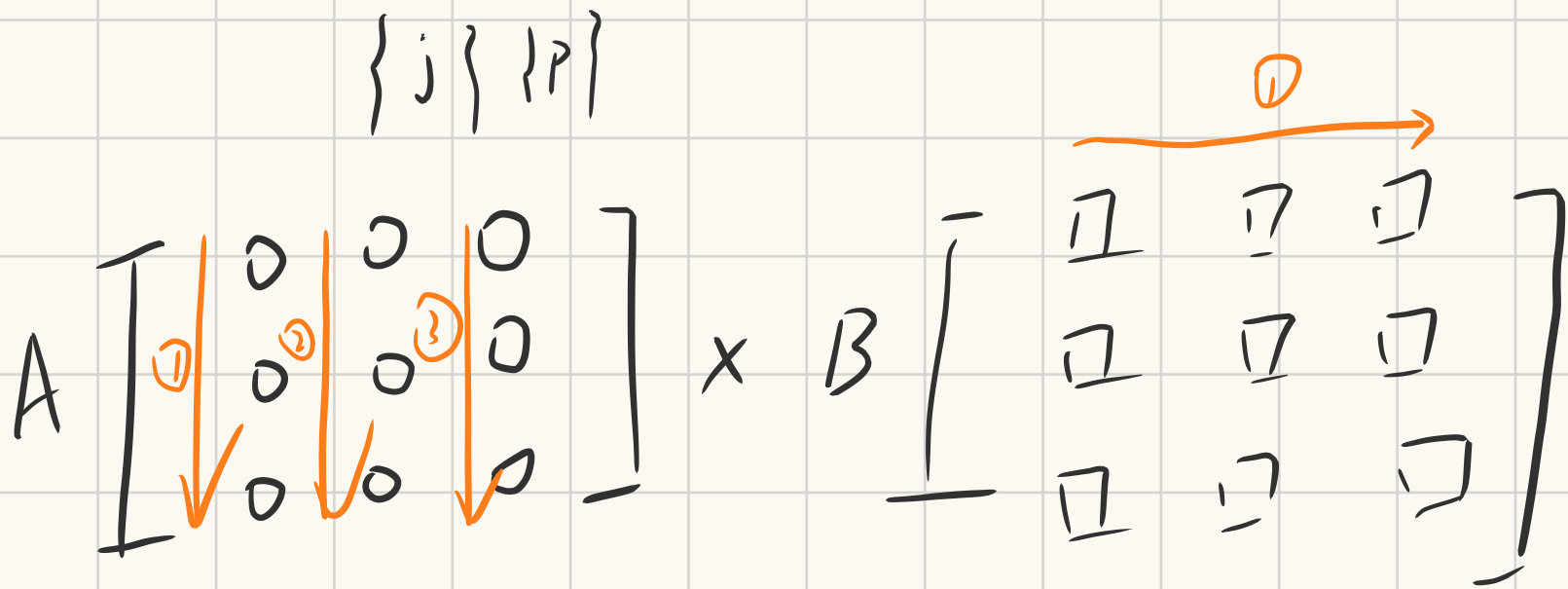
Optimization 1



对于 B 矩阵完全遍历

由于列序

B 的 cache miss 多



对于 A 矩阵完全遍历

由于列主序

B 的 (cache miss) 多

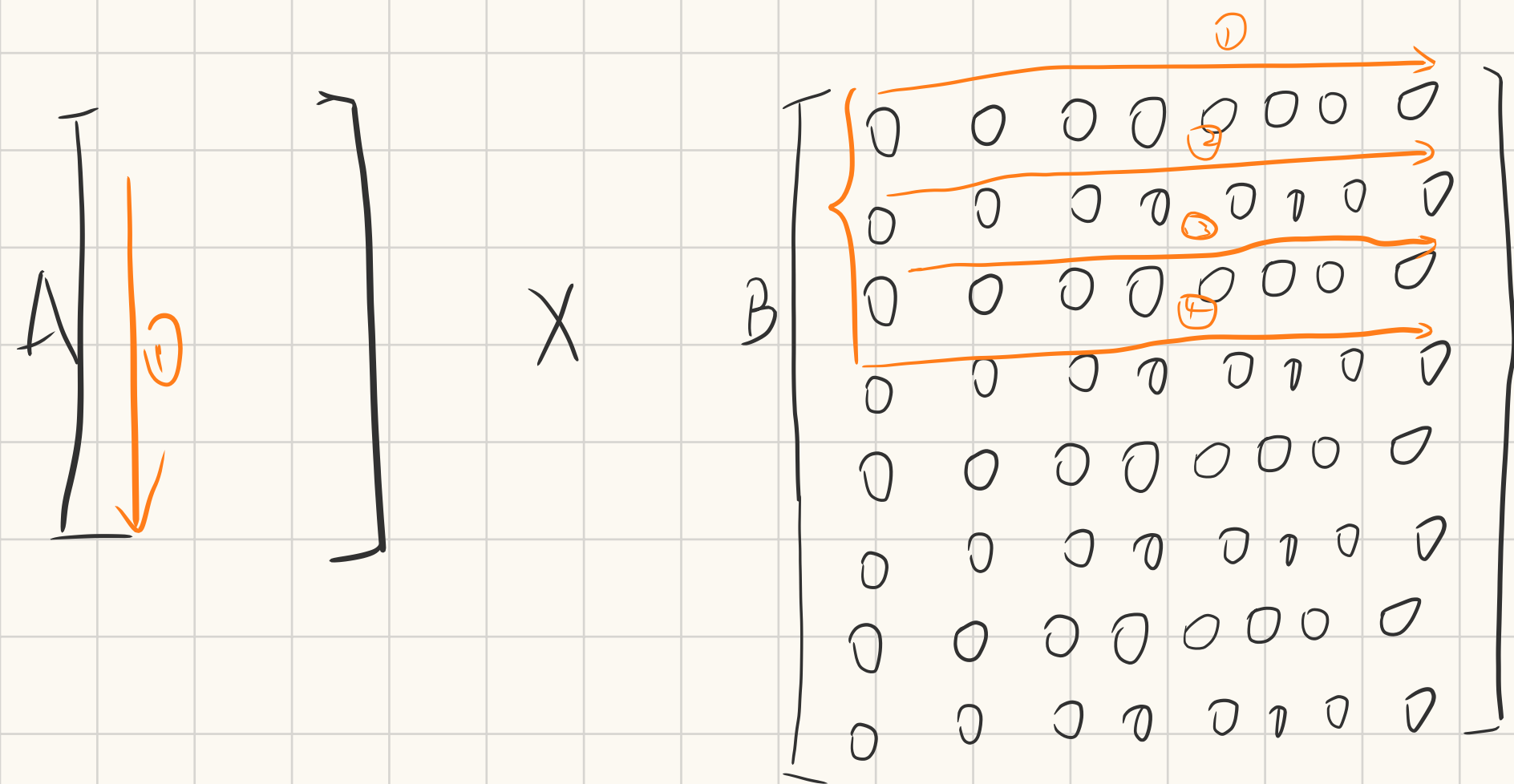
优化1只是改变乘法顺序

对于访存并没有优化, 所以性能无变化

optimization 2 (1x4) 只是进行循环展开

对于访存并未优化

所以仍然性能没有变化



四行B行在一个循环里处理

一次 $\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}^T \times \begin{bmatrix} y_1 & y_2 & y_3 & y_4 \\ y_1 & y_2 & y_3 & y_4 \\ y_1 & y_2 & y_3 & y_4 \\ y_1 & y_2 & y_3 & y_4 \end{bmatrix}^T$

1x4

optimization 3. (1x4)

增加1个函数

optimization 4 (1x4)

将函数内联

optimization 5 (1x4)

循环融合

① 只更新1次 P 完成4次运算

② 由于一次进行4次运算, cache line 读入后会立即使用 cache 效率高

Optimization 6 (1x4)

减少内存

将过程量放入寄存器

Optimization 7 (1x4)

用指针代替数组索引

原: $B(p, j) \Rightarrow B[j * (ldb) + p]$

现: $\begin{cases} ptr = B(0, j) \\ \text{loop } ptr++ \end{cases}$

能这样做的原因: 用一维数组存储,
B的读取是按行读取,
所以B的读取在内存上是连续的.

Optimization 8 (1x4)

负优化

循环展开

与编译优化可能限制编译器

Optimization 9 (1x4)

回掉寻址

优化不明显 $\Leftarrow$ 编译器自动优化

optimization $\rightarrow (4 \times 4)$

- 求函数 串行执行

$$A \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0^1 & 0^2 & 0^3 & 0^4 \\ 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} B$$

Diagram illustrating the matrix multiplication $A \times B$. Matrix A is a 4×4 matrix with elements $0, 0, 0, 0$ in the first three rows and $0^1, 0^2, 0^3, 0^4$ in the fourth row. Matrix B is a 4×4 matrix with elements $0, 0, 0, 0$ in the first three rows and $0, 0, 0, 0$ in the fourth row. Orange arrows indicate the dot product of the first row of A with the first column of B , labeled a . Similar arrows are shown for the second, third, and fourth rows of A , labeled b, c, d respectively.

$$\left\{ \begin{array}{llll} 1. a^T & 1. b^T & 1. c^T & 1. d^T \\ 2. a^T & 2. b^T & 2. c^T & 2. d^T \\ 3. a^T & 3. b^T & 3. c^T & 3. d^T \\ 4. a^T & 4. b^T & 4. c^T & 4. d^T \end{array} \right\}$$

运算过程

循环展开 $\Rightarrow$ 无明显变化

optimization

4

(4x4)

函数内联 $\Rightarrow$ 无明显变化

optimization

5

(4x4)

更新 1 次 P $\Rightarrow$ 运算 16 次

cache 中数据命中率 高

optimization

6

(4x4)

共用寄存器

但是 (4x4) 共用的 Reg 数量超出 CPU 实际数量
造成溢出 $\Rightarrow$ 不确定性

有明显提升

optimization 7 (4x4)

使用指针

减少计算, 访存变快 $\Rightarrow$ 有小幅提升

optimization 8 (4x4)

B 也利用寄存器 存储 复用数据

$\Rightarrow$ 单个 CPU 中 寄存器压力变大

$\Rightarrow$ 性能反而下降



optimization 9 (4x4)

改变循环中 计算 顺序

为后续 SIMD 做准备

Optimization 10 (4x4)

引入 SIMD

- 一次计算 2 行

Optimization 11 (4x4)

分块 [大规模数据分块]

为了使 Cache 命中率增加

数据 reuse

减少 内存带宽 带来的限制

Optimization 12 (4x4)

A 打包 [使 A 矩阵数据重构]

使得小分块中内存连续 $\Rightarrow$ 数据复用

optimization 13 (4x4)

一次打包

减少打包开销.

Add 101 4x4 中: $\left\{ \begin{array}{l} -mm-load-pd(a) \\ -mm-load-pd(a+2) \\ a+=4 \end{array} \right.$

体现了 pack 的效果.

optimization 14 (4x4)

optimization 15 (4x4)

B 打包

提高 B 矩阵的数据 reuse

提高 cache 命中率

总结:

1. 一次P 多次计算 $\Rightarrow$ 减少访存 提高cache命中
2. 过程量放入寄存器 指针代替索引
3. $1 \times 1 \rightarrow 1 \times 4 \rightarrow 4 \times 4 \Rightarrow$ 便于使用 SIMD
4. 分块 $\Rightarrow$ 使大规模数据 不访存受限
5. A B 分块 pack $\Rightarrow$ 使得内存连续 提高reuse